

Universidade do Minho

Algoritmos e Estruturas de Dados

Engenharia de Telecomunicações e Informática

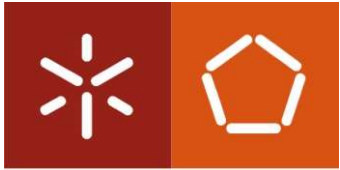
2022/2023

Telmo Adão

Departamento de Sistemas de Informação

Universidade do Minho

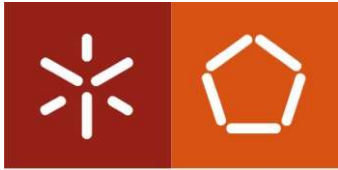
Slides baseados no material didático disponibilizado pelo Professor Luís Magalhães



Universidade do Minho

Sumário

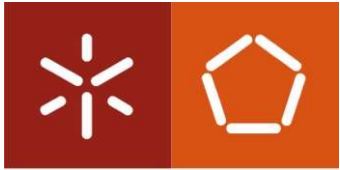
- Revisão de conceitos:
 - Análise de um problema
 - Fases de construção de uma solução
 - Testes em algoritmos
 - Revisão de conceitos de C
 - Exercícios



Breve revisão de conceitos

Universidade do Minho

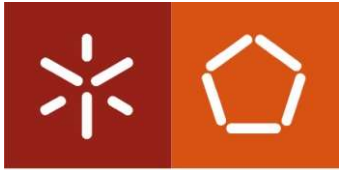
- Algoritmo: conceito e conceção (delineamento de um programa, independente da tecnologia)
- Conceitos gerais de programação (linguagem C)
- Preparação e execução de programas (algoritmo, conversão p/ programa e estrutura de um programa em C)
- Tipos de dados elementares (int, float, char, etc.)
- Operações: aritméticas e lógicas (+,-,*,/,% , ||, &&, ...)
- Estruturas de decisão e repetição (if..else if..else, switch..case, for, while, do..while)



Universidade do Minho

Breve revisão de conceitos

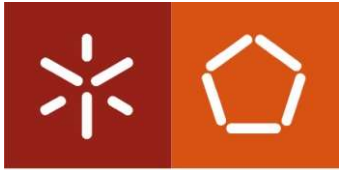
- Funções (parâmetros e escopo)
- Bibliotecas standard (string.h e math.h) e customizadas
- Vetores e matrizes
- Pesquisa linear e ordenação (e.g. bolha)
- Strings (char*) e ficheiros (leitura e escrita)
- Apontadores e Structs → muito importante para esta UC...



Universidade do Minho

Fases da resolução de um problema

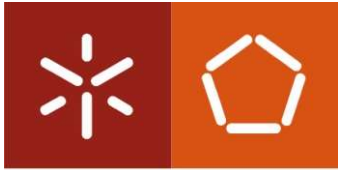
- Análise
 - Ler atentamente o texto de descrição do problema;
 - compreender clara e completamente o problema descrito
 - retirar do texto do enunciado os conjuntos de dados de entrada (input);
 - retirar do texto do enunciado os conjuntos de dados de saída (output);
 - retirar do texto do enunciado, e representar simbolicamente, as relações entre os conjuntos de dados de entrada e os de saída



Universidade do Minho

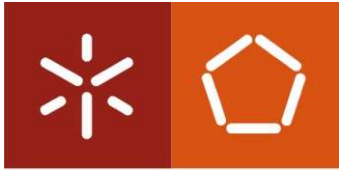
Fases da resolução de um problema

- Desenho
 - Definir as estruturas de dados a usar
 - Construir o algoritmo
- Codificação / Implementação
 - criação do programa com base no algoritmo
- Testes
 - descobrir os erros de execução: o programa não pode ser executado
 - descobrir os erros de algoritmo: os resultados foram calculados, mas estão errados



Algoritmo

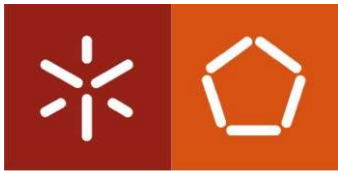
- Surge da necessidade de resolução de um problema
- Consiste na descrição de um conjunto de passos
- É independente da linguagem de programação
- Deve ser coerente e consistente – exemplo da ATM
 - Coerente: Permitir inserir o pin antes de colocar o cartão, é coerente?
 - Consistente: garantir que funciona sempre para as condições previstas!



Algoritmo [Exemplos]

Universidade do Minho

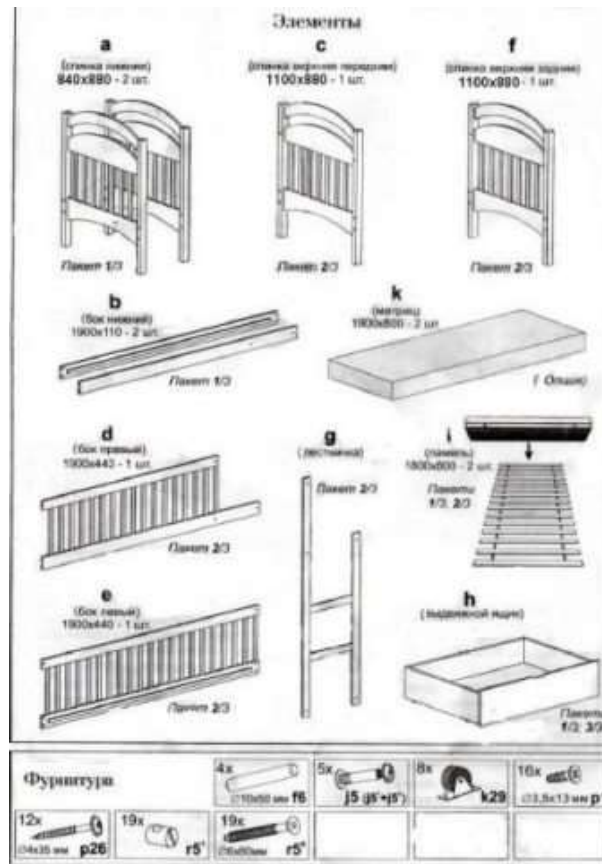
- Receita leite creme
 1. Preparar ingredientes: 1 l de leite meio-gordo, 7 gemas de ovo M, 150 g açúcar + qb para queimar, 2 c. sopa farinha Maizena qb, casca de limão.
 2. Colocar as gemas numa taça e misturar com uma vara de arames e lentamente adicione a farinha
 3. Colocar de seguida o açúcar e voltar a misturar, e por fim o leite frio
 4. Levar a mistura num tacho ao lume e juntar a casca de limão e um pau de canela previamente lavados
 5. Deixe engrossar em lume brando
 6. Quando engrossar, desligar o lume e reserve
 7. Verter a mistura numa taça grande ou distribua por taças individuais e polvilhar com açúcar queimado com um ferro próprio ou um maçarico de cozinha

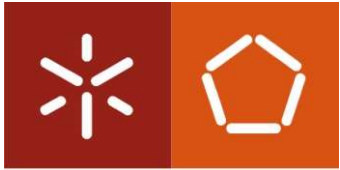


Algoritmo [Exemplos]

Universidade do Minho

Instruções IKEA

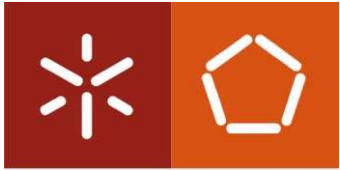




Algoritmo

Universidade do Minho

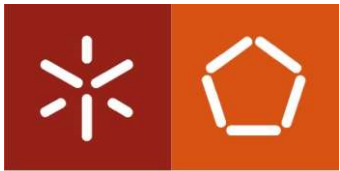
- Algoritmo: sequência de passos, ou operações elementares que aplicadas sobre determinada informação inicial permitem dela obter em tempo finito e de modo não ambíguo uma dada informação (resultado)
- Características desejáveis:
 - Correção
 - Clareza e simplicidade
 - Eficiência



Algoritmo

Universidade do Minho

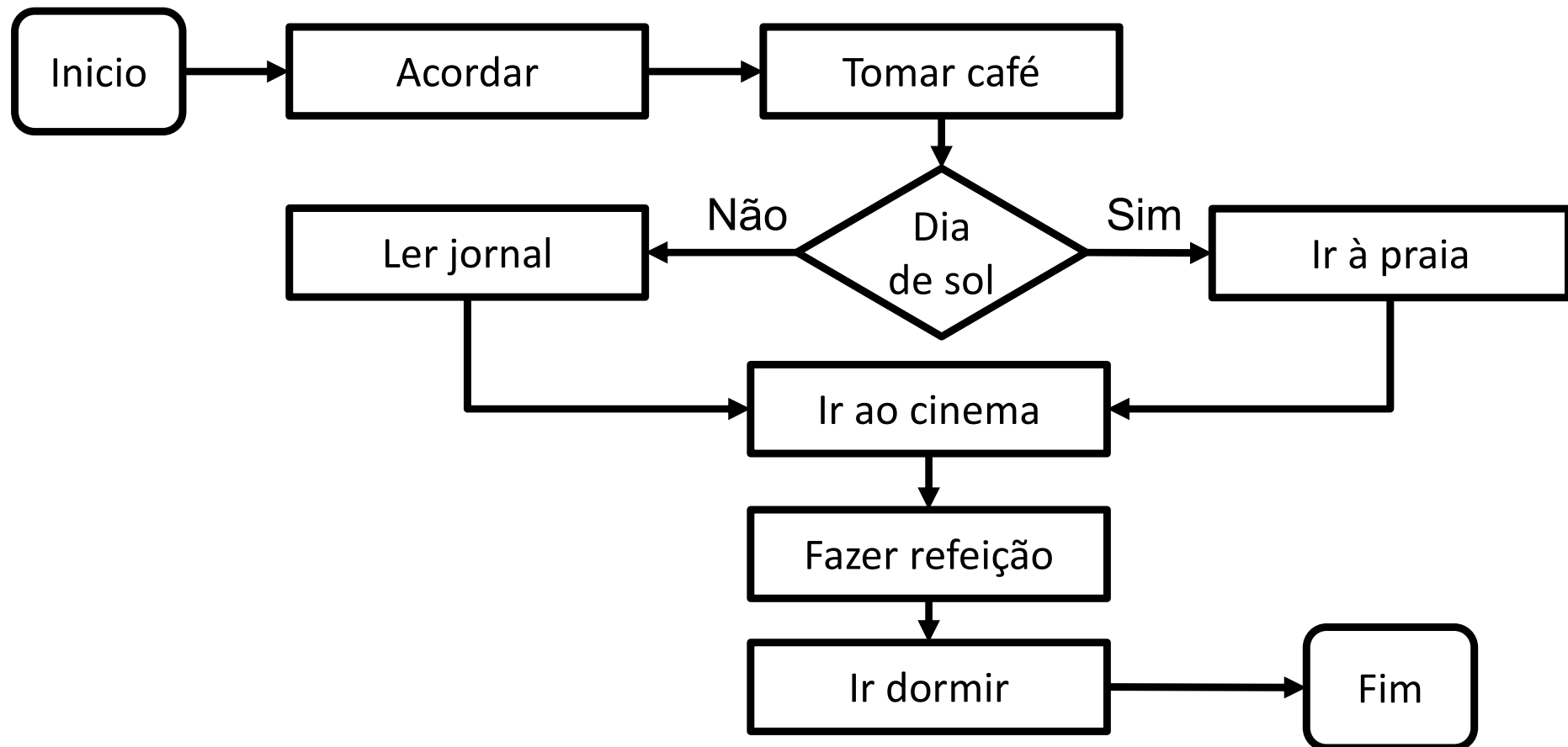
- Pseudo-código: linguagem descritiva
 - Algoritmo de domingo.
 - Acordar.
 - Tomar o café.
 - Se estiver sol vou à praia senão leio o jornal.
 - Almoçar.
 - Ir ao cinema.
 - Fazer uma refeição.
 - Ir dormir.
 - Final do domingo.

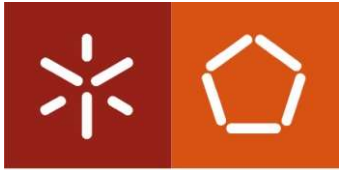


Algoritmo

Universidade do Minho

- Fluxograma: Linguagem gráfica

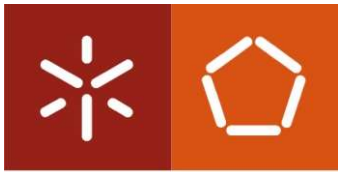




Universidade do Minho

Algoritmo

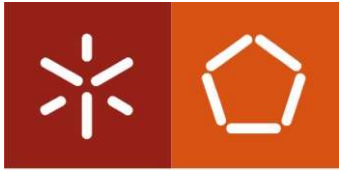
Escreva um algoritmo que descreva a sequência de passos necessários para mudar um pneu de um automóvel...



Teste dos algoritmos

- Teste ou *tracing* de um algoritmo/programa
 - seguir a execução de um algoritmo (ou programa) passo a passo e verificar a evolução de todas as variáveis
 - pretende-se testar um algoritmo para determinados valores de entrada, observando o seu comportamento
 - Este processo é apresentado através de uma tabela em que deve constar, pelo menos, todas as variáveis e saída de resultados

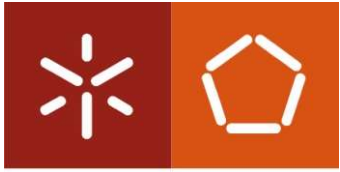
Passo	BASE	EXP	PRODUTO	CONTADOR	Teste	Output
1	?	?	?	?		
2	9	?	?	?		
3	9	4	?	?		
4	9	4	9	?		
5	9	4	9	1		
6	9	4	9	1	V	
7	9	4	81	1		
8	9	4	81	2		
6	9	4	81	2	V	
7	9	4	729	2		
8	9	4	729	3		
6	9	4	729	3	V	
7	9	4	6561	3		
8	9	4	6561	4		
9	9	4	6561	4	F	
10	9	4	6561	4		6561



Exercício

Universidade do Minho

- Pretende-se criar um programa para ler 3 valores, dados pelo utilizador, e indicar qual deles é o menor
 - Executar as fases da resolução de um problema
 - Fazer o tracing do programa

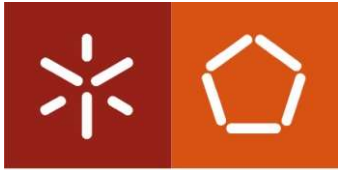


Linguagem de Programação C

Universidade do Minho

- Composto por uma ou mais funções
- Uma das funções tem de ser a função main, a qual só pode surgir uma vez
- Função main
 - Primeiro bloco de instruções a ser executado
- Exemplo mais simples

```
main()  
{  
}
```

Operadores aritméticos e lógicos

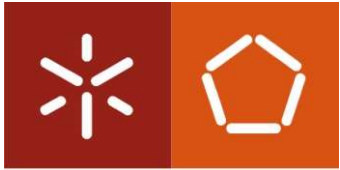
Universidade do Minho

- Operações aritméticas

Operação C	Operador Aritmético	Expressão algébrica	Expressão em C
Soma	+	$a+b$	$a + b$
Subtração	-	$a-b$	$a - b$
Multiplicação	*	ab ou $a \times b$	$a * b$
Divisão	/	a/b	a / b
Módulo	%	$a \bmod b$	$a \% b$

- Regras de precedência

Operador(es)	Operação(ões)	Ordem do cálculo (precedência)
()	Parentises	Prioridade máxima. Se os parênteses estiverem “aninhados”, a expressão no par mais interno será avaliada primeiro. Se houver vários pares de parênteses "no mesmo nível" (ou seja, não aninhados), eles serão avaliados da esquerda para a direita, por ordem de surgimento.
*, /, or %	Multiplicação, divisão, modulus	Prioridade de segunda ordem. Se houver vários, eles são avaliados da esquerda para a direita.
+ or -	soma, subtração	Avaliado por último. Se houver vários, eles são avaliados da esquerda para a direita.

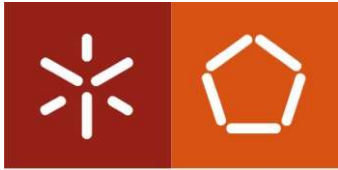


Oper. aritméticos e lógicos

Universidade do Minho

- Operadores relacionais e de igualdade

Operador algébrico de igualdade ou operador relacional	Correspondência em C	Exemplo de condição	Significado da condição
<u>Operadores de igualdade</u>			
=	==	$x == y$	x é igual a y (?)
≠	!=	$x != y$	x não é igual a y (?)
<u>Operadores relacionais</u>			
>	>	$X > y$	X é maior que y (?)
<	<	$X < y$	X é menor que y (?)
≤	>=	$X >= y$	X é maior que ou igual a y (?)
≥	<=	$X <= y$	X é menor que ou igual a y (?)

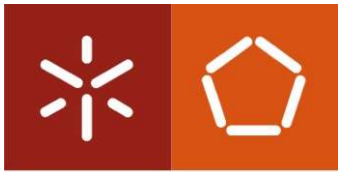


Entrada e saída de dados

Universidade do Minho

- O C não tem instruções / funções próprias para a entrada e saída de dados
- Recorre a bibliotecas de funções
 - Incluir no início: `#include <stdio.h>`
- Exemplo:

<code>#include <stdio.h></code>	Inclui também as funções de Entrada e Saída
<code>main()</code>	O Programa começa aqui
<code>{</code>	Início do Bloco de Instruções
<code>printf("Hello World");</code>	Escrever a <i>string</i> “Hello World” usando a função <code>printf</code>
<code>}</code>	Fim do Bloco de Instruções (e fim de programa)



Entrada e saída de dados

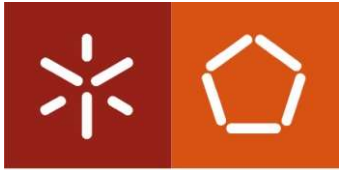
Universidade do Minho

- `printf`
 - `printf(format-control-string, other-arguments);`

- Algoritmo (pseudo-código)
 - *Escrever(x)*

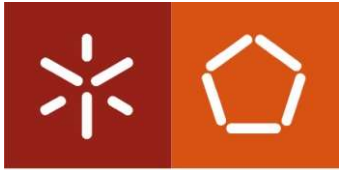
- Exemplos:
 - `printf("Exemplo");`
 - `printf("O valor é: %d", a);`

Tipo	Formato	Observações
<code>char</code>	<code>%c</code>	Um único caractere
<code>int</code>	<code>%d</code> ou <code>%i</code>	Um inteiro (Base d ecimal)
<code>int</code>	<code>%o</code>	Um inteiro (Base o ctal)
<code>int</code>	<code>%x</code> ou <code>%X</code>	Um inteiro (Base hexadecimal)
<code>short int</code>	<code>%hd</code>	Um s hort inteiro (Base d ecimal)
<code>long int</code>	<code>%ld</code>	Um l ong inteiro (Base d ecimal)
<code>unsigned short int</code>	<code>%hu</code>	short inteiro positivo
<code>unsigned int</code>	<code>%u</code>	inteiro positivo
<code>unsigned long int</code>	<code>%lu</code>	long inteiro positivo
<code>float</code>	<code>%f</code> ou <code>%e</code> ou <code>%E</code>	
<code>double</code>	<code>%f</code> ou <code>%e</code> ou <code>%E</code>	



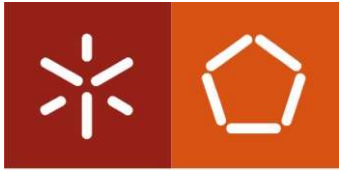
Entrada e saída de dados

- scanf
 - `scanf(format-control-string, other-arguments);`
- Algoritmo (pseudo-código)
 - *Ler(x)*
- Exemplos:
 - `scanf("%d", &a);`
 - `scanf("%f", &b);`



Exercícios

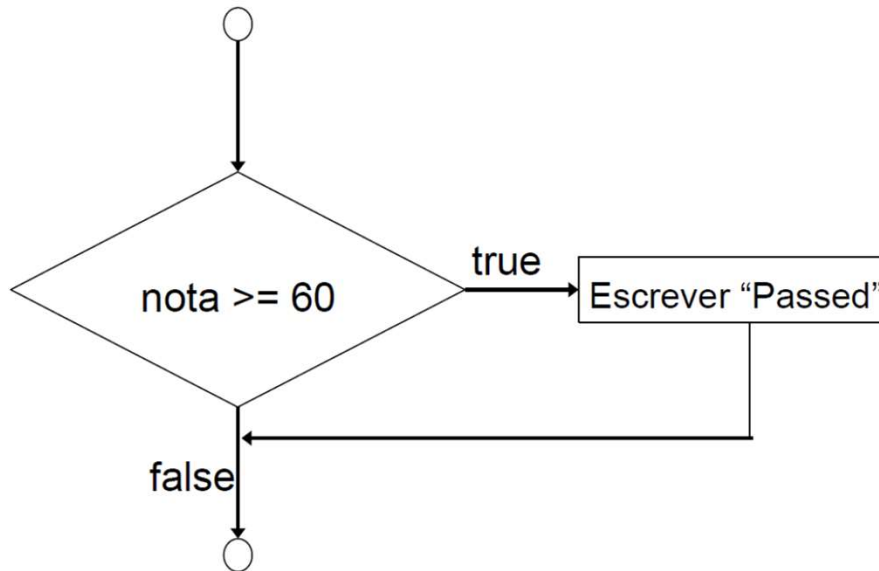
- Escreva um programa em C para calcular a soma de dois números
- Escreva um programa em C para calcular a média de três números
- Escreva um programa em C para calcular a área e o perímetro de um retângulo



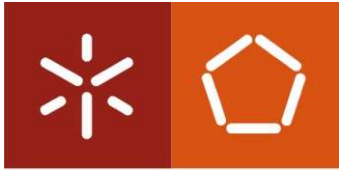
Estruturas condicionais - if

Universidade do Minho

- Estrutura If (Se), numa estrutura de entrada/saída única



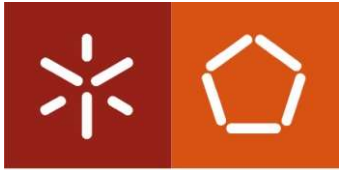
A decision can be made on any expression.
zero - false
nonzero - true
Example:
3 - 4 is true



Estruturas condicionais - if

Universidade do Minho

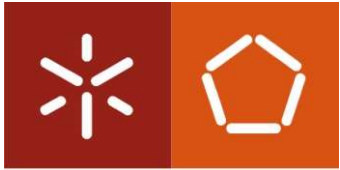
- If (“true”) → apenas realiza uma ação se a condição for verdadeira
- if...else “false” → especifica uma ação para ambas as situações de “true” e “false”
- Pseudocódigo:
Se nota ≥ 60 então
 Escrever “Aprovado”
senão
 Escrever “Reprovado”
Fim_se
- Atenção ao espaçamento/indentação



Estruturas condicionais - if

Universidade do Minho

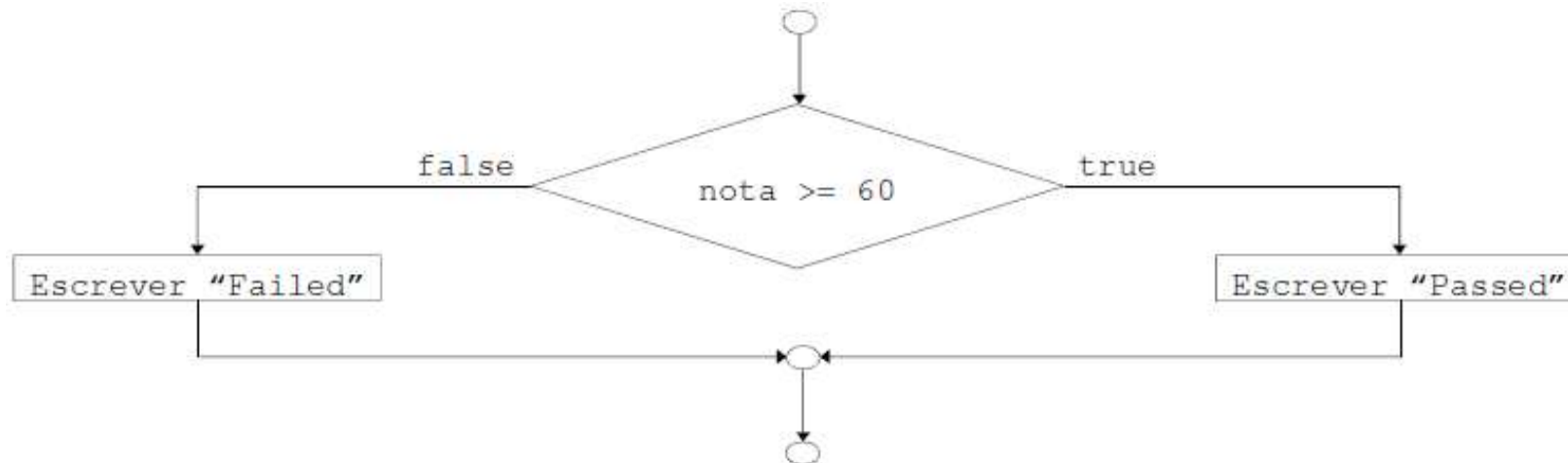
- Código em C:
if (nota>=60)
 printf("aprovado\n");
else
 printf("reprovado\n");
- Operador ternário (ternary conditional operator →
condition?true:false)
 - Leva 3 argumentos (condição, valor de *true* e valor de *false*)
 - Pode ser escrito assim: printf("%s\n", nota>= 60 ? "aprovado" : "reprovado");
 - Ou então assim: nota >= 60 ? printf("aprovado") : printf("reprovado");



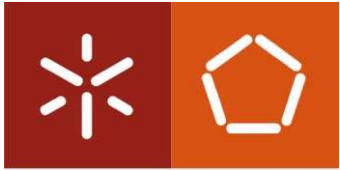
Estruturas condicionais - if

Universidade do Minho

- Fluxograma de uma estrutura de decisão **if..else**



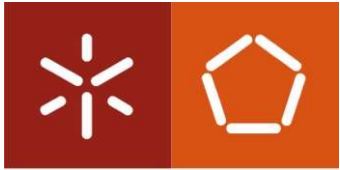
- if.. else** “aninhado”
 - Teste para vários casos colocando instruções de seleção **if..else** dentro de outras estruturas **if..else**
 - Depois que a condição é atendida, as restantes instruções são ignoradas
 - Recuo profundo geralmente não usado na prática



Estruturas condicionais - if

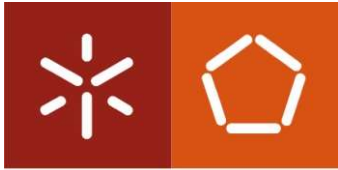
Universidade do Minho

- Valores lógicos
 - Falso → em ANSI C: 0
 - Verdadeiro → em ANSI C: todo o valor diferente de 0
- Operadores relacionais
 - ==, >, <, >=, <=, !=
 - Nota: retorna um valor falso (0) ou verdadeiro (1)
- Operadores lógicos
 - &&, ||, !



Exercícios

- Pedir um valor ao utilizador e indicar se é positivo ou negativo.
- Calcular o valor absoluto de um número indicado pelo utilizador
- Dados dois valores (pelo utilizador) indicar qual é o maior
- Dados três valores (pelo utilizador) indicar qual é o maior introduzido
- Analise a complexidade das soluções anteriores recorrendo à notação O-grande



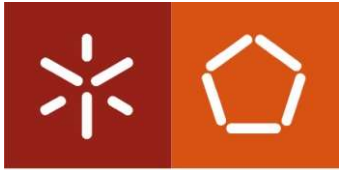
Estruturas condicionais - switch

Universidade do Minho

- **Switch**: útil para quando uma variável ou expressão é testada para todos os valores que pode assumir e ações que podem ser tomadas
- Formato: conjunto de casos variáveis + caso por “default”

```
switch ( value ){  
    case '1':  
        actions  
    case '2':  
        actions  
    default:  
        actions  
}
```

- *Value* → é um valor numérico do tipo char, int ou long...
- *Break* → quebra o switch



Estruturas condicionais - switch

Universidade do Minho

- Pseudo-código

Caso **condição_1**:

 instrução(ões)_1

condição_2:

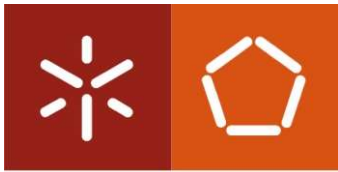
 instrução(ões)_2

 ...

CASO_CONTRÁRIO

 instrução(ões)

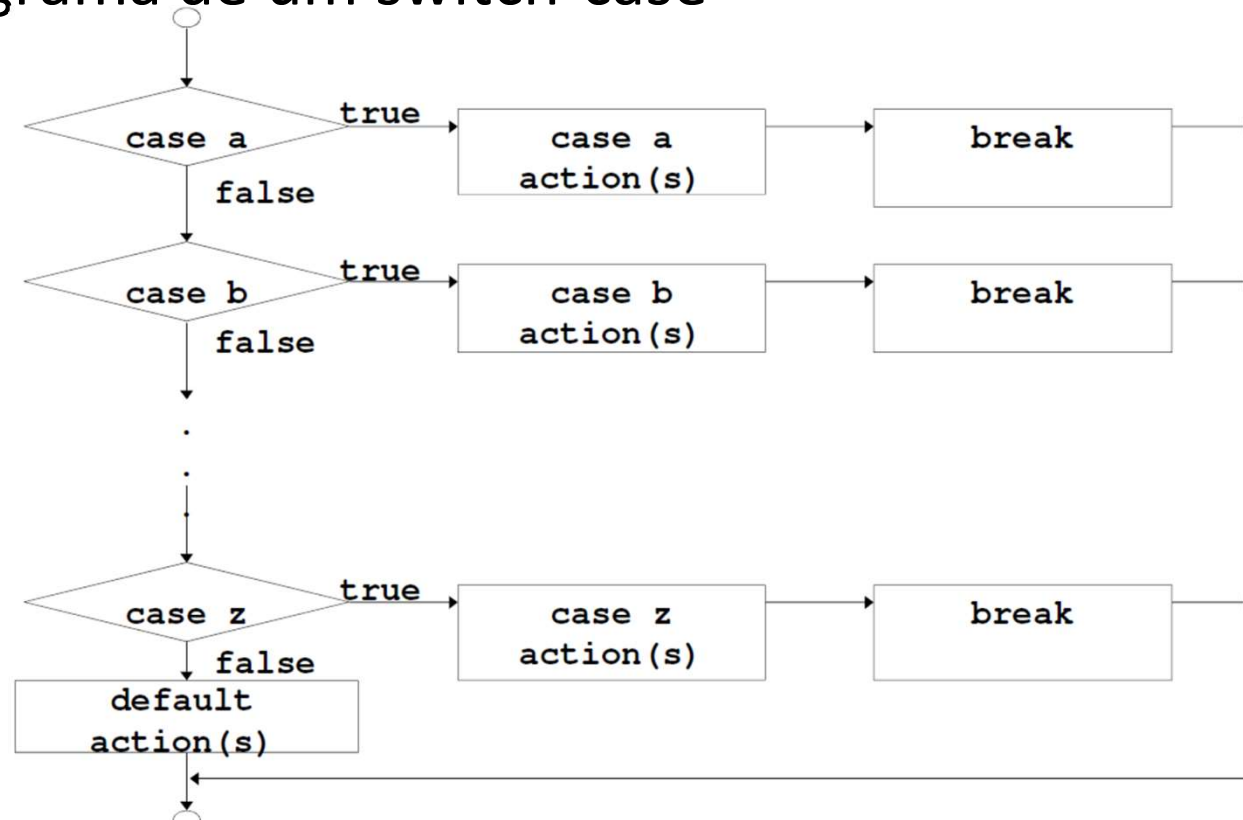
Fim_caso

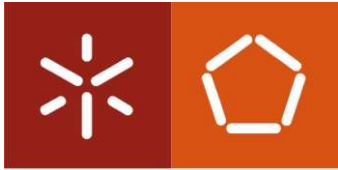


Estruturas condicionais - switch

Universidade do Minho

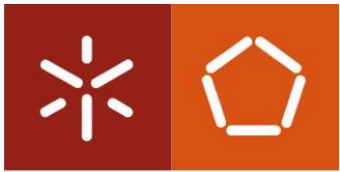
- Fluxograma de um switch-case





Exercícios

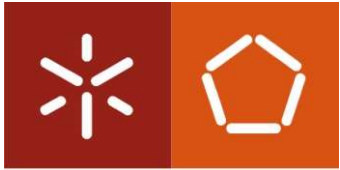
- Escreva um programa que calcule o valor de uma expressão binária escrita pelo utilizador
- Escreva um programa que calcule o salário final de um funcionário. Os funcionários mediante a categoria a que pertencem pagam o seguinte imposto: 1 – 10%, 2 – 20%, 3 – 25%.
- Pedir dois números inteiros ao utilizador e indicar se algum deles é múltiplo do outro.
- Pedir 5 valores ao utilizador e calcular a média dos números. Caso um número seja negativo, não será considerado para o cálculo da média
- Analise a complexidade das soluções anteriores recorrendo à notação O-grande



Universidade do Minho

Operadores de atribuição

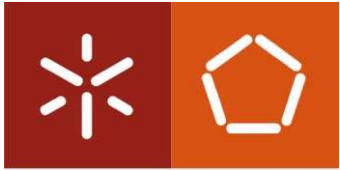
Operador	expressão (curta)	Forma comprida
++	++a	Primeiro incrementa, depois atualiza “a” (devolve valor já com incremento)
++	a++	Devolve o valor atual de “a” e só depois o incrementa
--	--b	Primeiro decrementa, depois atualiza “b” (devolve valor já com decremento)
--	b--	Devolve o valor atual de “b” e só depois o decrementa



Estruturas de Controlo

Universidade do Minho

- Estrutura de repetição
 - O programador especifica ações que são repetidas enquanto determinada condição se verificar como **verdadeira**
 - A estrutura ***while*** repete até que a condição se torne **falsa**



Universidade do Minho

Estruturas de Controlo

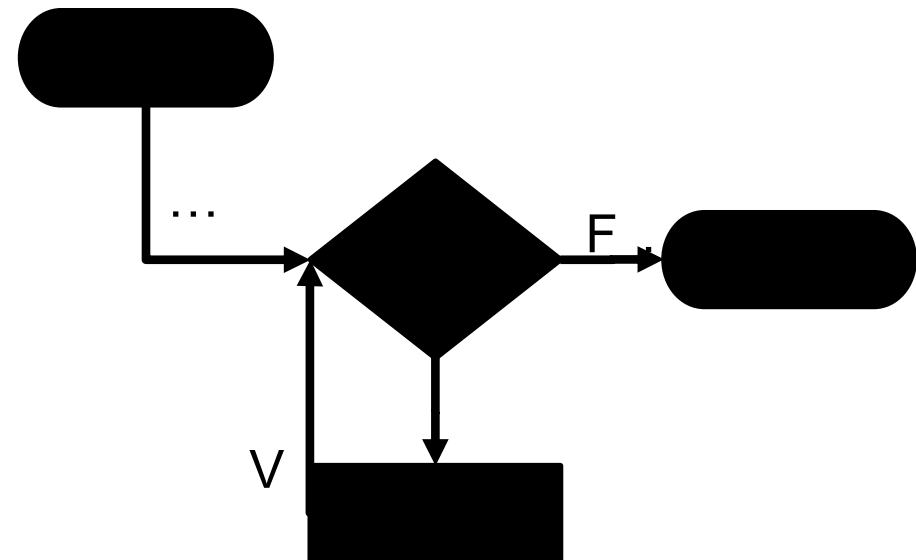
Repetição em pseudo-código

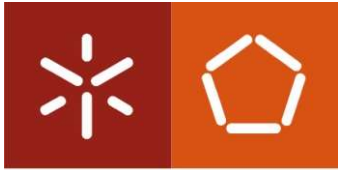
ENQUANTO condição FAZER

Instrução(ões)...

FIM_ENQUANTO

Repetição em fluxograma





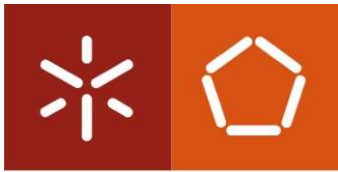
Estrutura de repetição **do..while**

Universidade do Minho

- O bloco **do..while**
 - É similar ao **while** já visto
 - A condição de paragem é avaliada depois do corpo do *loop* (executa pelo menos uma vez)
 - Formato:

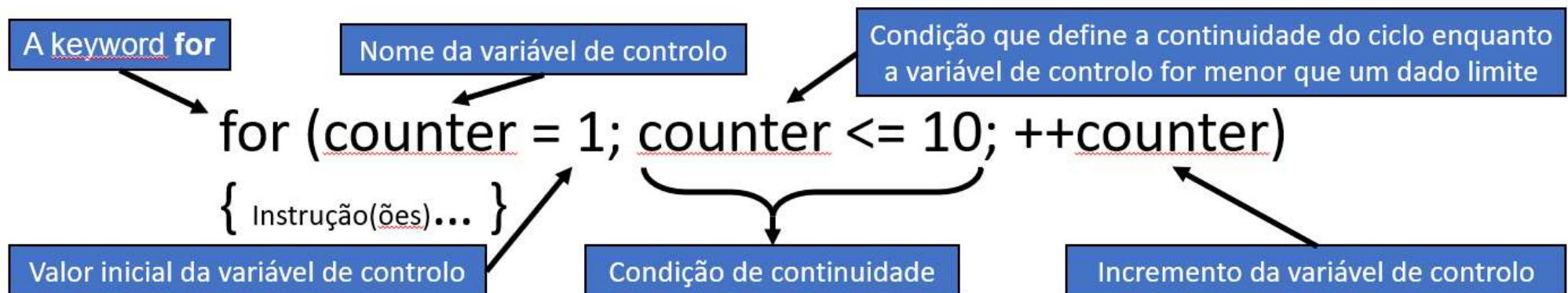
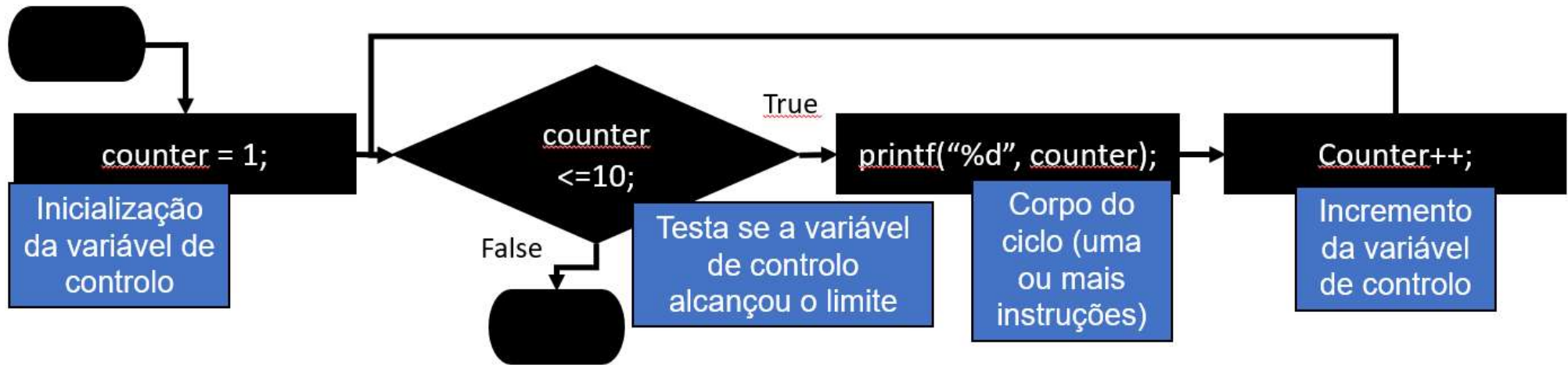
```
do {  
    instruções;  
} while ( condição );
```
- Exemplo (considerando a variável *counter = 1*):

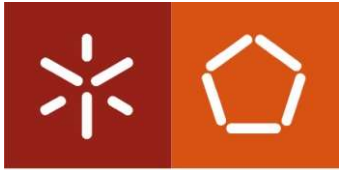
```
do {  
    printf( "%d ", counter );  
} while (++counter <= 10);
```
- Imprime números inteiros de 1 a 10



O ciclo For (Fazer..Para)

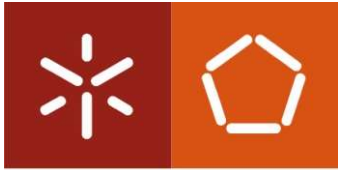
Universidade do Minho





Exercícios

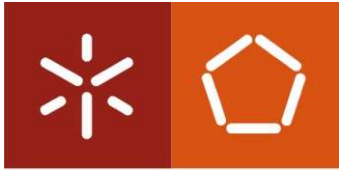
- Escreva um algoritmo e um programa em C para calcular a média de n números (agora use a estrutura for).
- Escreva um algoritmo e um programa em C para dados n números, indicados pelo utilizador, calcular a soma dos números impares e indicar quantos números positivos foram introduzidos pelo utilizador.
- Analise a complexidade das soluções anteriores recorrendo à notação O-grande



Universidade do Minho

As instruções *break* e *continue* (revisão)

- Instrução *break*
 - Causa a saída imediata de um ciclo (for, while, do..while) ou switch
 - A execução do programa continua com a primeira instrução imediatamente a seguir à estrutura quebrada
 - Usos comuns:
 - Saltar as restantes alternativas de um *switch*
 - Quebrar um ciclo antes da condição de fim deixar de ser verificada (termina mais cedo)



Universidade do Minho

As instruções *break* e *continue* (revisão)

- Instrução *continue*
 - Ignora as restantes instruções *dentro* do corpo de um ciclo (for, while, do..while) → avança imediatamente para a próxima iteração do ciclo
 - *While* e *do..while*: a condição de paragem do ciclo é testada imediatamente a seguir à execução da instrução *continue*
 - *For*: o incremento é executado e só depois é que a condição de paragem é avaliada